

MAJOR PROJECT REPORT ON
FUTURE OF LOAN APPROVAL WITH EXPLAINABLE AI
Submitted to
SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY
In Partial fulfilment of the requirement for the award of Degree of
BACHELOR OF TECHNOLOGY
IN
DATA SCIENCE

By

T. VARSHITHA (22D41A67B2)
P. HARI KRISHNA (22D41A6789)
S. SHIVA SAI (22D41A67A8)
Y. PRASUNA (22D41A67C5)
T. KOUSHIK (23D45A6711)

Under the Guidance of

Ms. B.K.N.PRIYANKA

(Assistant Professor)



DEPARTMENT OF DATA SCIENCE
SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution under UGC, Accredited by NBA, Affiliated to JNTUH)

Sheriguda, Ibrahimpatnam, RR District – 501510

(2025-2026)

SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution under UGC, Accredited by NBA, Affiliated to JNTUH)

DEPARTMENT OF DATA SCIENCE



CERTIFICATE

This is to certify that the project work report entitled “**FUTURE OF LOAN APPROVALS WITH EXPLAINABLE AI**” submitted by **T.VARSHITHA (22D41A67B2), P.HARI KRISHNA (22D41A6789), S.SHIVA SAI (22D41A67A8), Y.PRASUNA (22D41A67C5), T.KOUSHIK (23D45A6711)** in Partial fulfilment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY in DATA SCIENCE, SRI INDU COLLEGE OF ENGINEERING AND TECHNOLOGY, HYDERABAD** is a record of Bonafide work carried out by them under our guidance and supervision.

The results presented in this thesis have been verified and are found to be satisfactory. The results embodied in this thesis have not been submitted to any other university for the award of any other Degree or Diploma.

GUIDED BY

Ms. B.K.N.PRIYANKA

(Assistant Professor)

Prof. SURESH BALLALA

(Head of the Department)

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of the task would be put incomplete without the mention of the people who made it possible, whose constant guidance and encouragement grown all the efforts with success.

First and foremost, we extend our thanks to our chairman **SRI R. VENKATA RAO** sir for providing the vision and resources necessary for this endeavour. Your leadership and commitment to excellence have been invaluable source of inspiration.

We are also thankful to Principal **Dr. G.SURESH** sir for giving us the permission to carry out this project and for providing necessary infrastructure and labs.

We are highly indebted to **SURESH BALLALA**, Professor and Head of the Department of **DATA SCIENCE**, for providing valuable guidance at every stage of this project.

We are grateful to our internal project guide **Ms. B.K.N.PRIYANKA**, Assistant Professor for her constant motivation and guidance given by her during the execution for this project work. We would like to thank the Teaching & Non-Teaching staff of Department of Data Science for sharing their proper knowledge with us, last but not least we express our sincere thanks to everyone who helped us directly or indirectly who helped in completing this project.

T. Varshitha	(22D41A67B2)
P. Hari Krishna	(22D41A6789)
S. Shiva Sai	(22D41A67A8)
Y. Prasuna	(22D41A67C5)
T. Koushik	(23D45A6711)

ABSTRACT

Widespread adoption of automated decision making by artificial intelligence (AI) is witnessed due to specular advances in computation power and improvements in optimization algorithms especially in machine learning (ML). Complex ML models provide good prediction accuracy; however, the opacity of ML models does not provide sufficient assurance for their adoption in the automation of lending decisions. This paper presents an explainable AI decision-support-system to automate the loan underwriting process by belief-rule-base (BRB). This system can accommodate human knowledge and can also learn from historical data by supervised learning. The hierarchical structure of BRB can accommodate factual and heuristic rules. The system can explain the chain of events leading to a decision for a loan application by the importance of an activated rule and the contribution of antecedent attributes in the rule. A business case study on automation of mortgage underwriting is demonstrated to show that the BRB system can provide a good trade-off between accuracy and explainability. The textual explanation produced by the activation of rules could be used as a reason for denial of a loan. The decision-making process for an application can be comprehended by the significance of rules in providing the decision and contribution of its antecedent attributes.

TABLE OF CONTENTS

1.	INTRODUCTION.....	1
2.	LITERATURE SURVEY	2
3.	EXISTING SYSTEM.....	4
4.	PROPOSED SYSTEM.....	6
	4.1 PROPOSED SYSTEM ADVANTAGES	7
5.	SYSTEM ANALYSIS	8
	5.1 SYSTEM REQUIREMENTS	9
	5.2 SYSTEM IMPLEMENTATION	9
6.	UML DIAGRAMS	11
	6.1 USE CASE DIAGRAM:	11
	6.2 CLASS DIAGRAM:	12
	6.3 COLLABORATIVE DIAGRAM:	12
	6.4 SEQUENCE DIAGRAM:	13
7.	SYSTEM ENVIRONMENT.....	14
	7.1 WHAT IS PYTHON?	14
	7.2 ADVANTAGES OF PYTHON.....	15
	7.3 DISADVANTAGES OF PYTHON.....	16
8.	WHAT IS MACHINE LEARNING?	18
	8.1 CHALLENGES IN MACHINE LEARNING	20
	8.2 APPLICATIONS IN MACHINE LEARNING	20
	8.3 ADVANTAGES IN MACHINE LEARNING	21
	8.4 DISADVANTAGES OF MACHINE LEARNING	22
9.	MODULES USED IN PROJECT	23
10.	HOW TO INSTALL PYTHON ON WINDOWS AND MAC?	25
11.	SYSTEM TESTING.....	31
	11.1 TYPES OF TESTS.....	31
	11.2 TEST STRATEGY AND APPROACH.....	32
12.	SCREENSHOTS	34
13.	SOURCE CODE.....	40
14.	CONCLUSION	64
15.	REFERENCE	65

LIST OF FIGURES

S.NO	NAME OF THE FIGURE	PAGE NO.
1.	USE CASE DIAGRAM	11
2.	CLASS DIAGRAM	12
3.	COLLABORATIVE DIAGRAM	12
4.	SEQUENCE DIAGRAM	13

1. INTRODUCTION

Underwriting skill is learnt through several months of training and exchange of knowledge by senior underwriters. This task requires underwriters to be fairly analytical, very organized, and accurate to give informed decision to approve or reject a loan application. Underwriters concurrently analyse a large quantity of information to find affordability, repayment history and collateral. Furthermore, sometimes they are required to change the process due to a shift in regulatory and compliance standards, investor requirements, and customer demands (Krovvidy, 2008).

New technology and strong machine learning (ML) algorithms have opened the doors for a straight through loan application process. Artificial intelligence (AI) systems can execute rules and process customers' information in a few milliseconds. Financial institutions have recognized the benefits of AI and are using it in a different subset of the underwriting process and are keen to test and implement newly introduced digital innovation. AI systems are expected to replicate human decision-making skills. However, even today transformation of various algorithmic concepts into training data could be very challenging to solve every instance of the problem for a range of lending products. It may not be able to solve a tiny subset of the problem (Aggour, Bonissone, Cheetham, & Messmer, 2006).

Manual Underwriting Challenges: Manual underwriting task is a very much paper-based process. It is an inconvenient process of circulation of loan application files within different departments of a lending institution. Full attention to details is requisite to give sound judgment on an application. Human underwriter evaluates scenarios by analysing a large amount of dynamic information in a loan application. This could be a source of inconsistency, inaccuracy, and biases (Peterson, 2017).

Manual underwriting is often successful in processing non-standard loans. Many lenders like high street banks and building societies follow strict rules and do not offer personalized underwriting. However, there are some lenders who exercise common sense lending approaches for assessing both standard and non-standard cases such as non-standard properties, non-standard income/employment, and less than perfect credit scores. The underwriting process of non-standard cases is very detailed and individualistic. Common senses lending approaches serve underserved people

2. LITERATURE SURVEY

Title: “Explainable AI (XAI) in Financial Decision Making”:

Authors: Miller, Tim. et al.

Description: This seminal work introduces the concept of Explainable AI and its importance in the financial domain. It highlights the need for interpretable models, especially in applications like loan underwriting, where transparency is critical.

Title: Interpretable Machine Learning for Credit Scoring: A Case Study on Peer-to-Peer Lending:

Authors: Ribeiro, Marco Tulio. et al.

Description: The paper discusses the application of interpretable machine learning models in credit scoring, emphasizing the importance of understanding model predictions. The study showcases how interpretability can be achieved without sacrificing predictive performance.

Title: Explainable Artificial Intelligence (XAI) in Credit Scoring: A Comprehensive Review:

Authors: Chen, Liang. et al.

Description: This review provides an extensive overview of the existing literature on XAI in credit scoring. It covers various approaches and methodologies employed to make credit scoring models more transparent and interpretable.

Title: Automated Loan Underwriting using Explainable Neural Networks:

Authors: Gupta, Ankush. et al.

Description: The paper introduces a neural network-based approach to automate loan underwriting. It focuses on the integration of interpretability techniques, making the neural network decisions understandable for stakeholders.

Future of Loan Approvals with Explainable AI

Title: Explainable Artificial Intelligence (XAI) for Credit Risk Management: A Case Study in Peer-to-Peer Lending:

Authors: Verma, Saurabh. et al.

Description: This case study explores the application of XAI in credit risk management, specifically in peer-to-peer lending platforms. It discusses the implementation of interpretable models to enhance trust among lenders and borrowers.

Title: Transparent Credit Scoring: The Potential of Feature Selection:

Authors: Baesens, Bart. et al.

Description: The paper investigates the role of feature selection in creating transparent credit scoring models. It explores how selecting relevant features can lead to more interpretable models without compromising accuracy.

Title: Explainable Machine Learning Models in Credit Scoring: A Practical Approach:

Authors: Kim, Hyunwoong. et al.

Description: This paper provides a practical approach to implementing explainable machine learning models in credit scoring. It discusses the challenges and opportunities associated with the integration of interpretable models in real-world applications.

Title: Trust in Automation: Integrating Ethical Considerations in AI for Credit Scoring:

Authors: Martin, Andrew. et al.

Description: Focusing on the ethical implications of AI in credit scoring, this paper discusses the importance of trust in automated decision-making systems. It emphasizes the need for transparency and ethical considerations in the development and deployment of AI models for loan underwriting.

3. EXISTING SYSTEM

In the existing loan approval system, banks and financial institutions rely on either manual evaluation by loan officers or conventional automated systems based on predefined business rules and basic machine learning models. These systems collect applicant data such as age, income, employment status, credit score, past repayment history, existing liabilities, and account activity. Based on this information, a decision is made to approve or reject the loan.

Many modern banks use machine learning models like logistic regression, decision trees, or neural networks to speed up the process and reduce human workload. However, most of these models function as **black-box systems**, meaning they provide only a final decision (approved/rejected) without explaining the reasoning behind it. As a result, neither customers nor bank employees can clearly understand which factors influenced the decision and how much each factor contributed.

Additionally, these systems are heavily dependent on historical data. If the historical data contains biased or outdated patterns, the system may continue making unfair or inaccurate decisions. Since financial regulations increasingly demand transparency and accountability, the lack of interpretability in current systems creates legal and ethical challenges for banks.

3.1 EXISTING SYSTEM DISADVANTAGES

1. Lack of Transparency:

The system does not explain why a loan is approved or rejected, making the decision process unclear to users and bank staff.

2. Bias in Decision-Making:

Models trained on historical data may reflect past discrimination, leading to unfair treatment of certain social or economic groups.

3. Low Customer Trust:

Applicants who receive rejections without proper reasons often feel dissatisfied and lose confidence in the bank's digital services.

4. Difficult Regulatory Compliance:

Financial laws require justification of automated decisions, but black-box models make it hard to generate legally acceptable explanations.

5. Limited Error Detection and Debugging:

When the model produces incorrect or illogical outcomes, it is difficult to trace the exact cause due to hidden internal logic.

6. Rigid and Inflexible Rules:

Rule-based systems cannot adapt well to new financial patterns or changing customer behaviors, reducing long-term effectiveness.

Machine learning models frequently generate a large number of false alerts, flagging legitimate transactions as suspicious. This increases the workload for compliance analysts, raises operational costs, and may negatively impact customer experience.

4. PROPOSED SYSTEM

The proposed system integrates machine learning with Explainable AI (XAI) techniques to create a transparent, fair, and accountable loan approval framework. Instead of only producing a final approval or rejection result, the system also provides human-understandable explanations for every decision.

The system first preprocesses applicant data by handling missing values, normalizing financial attributes, and encoding categorical features like employment type and marital status. A machine learning model (such as Random Forest, Gradient Boosting, or Neural Networks) is then trained on this data to predict loan approval outcomes.

To make the model interpretable, XAI techniques like *SHAP (SHapley Additive explanations)* or *LIME (Local Interpretable Model-Agnostic Explanations)* are applied. These methods analyze the trained model and assign importance scores to each input feature, showing how much each factor (e.g., income, credit score, debt ratio) contributed to the final decision. The explanations are displayed in a simple visual or textual format for both applicants and bank officers.

Additionally, the system includes bias detection modules that continuously monitor fairness metrics across different demographic groups. This ensures that the system does not disproportionately reject or approve loans for specific categories of users. By combining transparency, fairness, and predictive accuracy, the proposed system aligns with modern regulatory standards and enhances customer trust.

4.1 PROPOSED SYSTEM ADVANTAGES

1. High Transparency:

The system provides clear explanations of which features influenced the decision, making the process understandable and trustworthy.

2. Bias Detection and Fairness:

Explainable AI helps identify biased patterns in training data and predictions, allowing corrective actions to ensure equal treatment.

3. Improved Customer Trust and Satisfaction:

Applicants receive meaningful reasons for approvals or rejections, which improves confidence in the bank's decision-making process.

4. Regulatory Compliance and Legal Safety:

Interpretable decision reports can be generated to justify outcomes, helping banks comply with financial regulations.

5. Better Model Monitoring and Debugging:

Bank analysts can easily detect errors or illogical predictions by analyzing feature contributions and model behavior.

6. Personalized Feedback for Applicants:

The system suggests actionable improvements (e.g., reduce debt, increase savings), helping users improve future eligibility.

7. Adaptive and Scalable System:

The model can be retrained with new data, allowing it to adapt to changing financial trends and customer behaviors.

5. SYSTEM ANALYSIS

The existing loan approval system used by most financial institutions is largely based on traditional rule-based approaches and conventional machine learning models. These systems evaluate loan applications using predefined criteria such as credit score, income level, employment status, and past repayment history. While these methods are effective in automating the approval process, they lack transparency in decision-making. The models function as black-box systems, meaning that neither customers nor bank officials can clearly understand the reasoning behind loan approval or rejection decisions. This lack of interpretability reduces user trust, makes it difficult to identify bias in decisions, and creates challenges in meeting regulatory requirements that demand clear explanations for financial decisions.

Additionally, the existing system may unintentionally incorporate bias present in historical data, leading to unfair treatment of certain applicants. Customers who are rejected often receive no meaningful feedback, preventing them from understanding how to improve their eligibility in the future. Manual verification and limited explainability also increase operational inefficiencies and customer dissatisfaction, making the current system less adaptable to modern banking needs.

The proposed system introduces Explainable Artificial Intelligence (XAI) into the loan approval process to address these challenges. In this system, AI models not only predict whether a loan should be approved or rejected but also provide clear explanations for their decisions. Techniques such as feature importance analysis, SHAP, or LIME are used to identify and present the key factors influencing each decision. This ensures that both applicants and bank officials can understand the logic behind the outcome, improving transparency and accountability.

By offering explainable results, the proposed system enhances fairness by identifying potential biases and supporting ethical decision-making. It increases customer trust by clearly communicating the reasons for approval or rejection and provides actionable insights that help applicants improve their financial profiles. Furthermore, the system aligns with regulatory standards by maintaining clear audit trails and justifiable decisions, making it more reliable, efficient, and suitable for the future of automated loan approvals.

5.1 SYSTEM REQUIREMENTS

HARDWARE REQUIREMENTS:

- **System:** Pentium IV 2.4 GHz.
- **Hard Disk:** 40 GB.
- **RAM:** 512 Mb.

SOFTWARE REQUIREMENTS:

- **Operating system:** Windows.
- **Coding Language:** Python.

5.2 SYSTEM IMPLEMENTATION

1. **Data Preprocessing:** Prepare the textual data by removing noise, such as special characters, punctuation, and stop words. Tokenize the text into sentences or paragraphs to facilitate sentiment analysis and summarization.
2. **Sentiment Analysis Model:** Implement or utilize pre-trained sentiment analysis models capable of accurately detecting the sentiment polarity (positive, negative, neutral) of each sentence or paragraph in the text. Consider employing advanced techniques such as deep learning-based models or transformer architectures for improved accuracy.
3. **Summarization Model:** Implement a text summarization model capable of generating concise summaries while incorporating sentiment information. Explore both extractive and abstractive summarization techniques, considering factors such as coherence, informativeness, and sentiment preservation.
4. **Integration:** Integrate the sentiment analysis module with the summarization module to leverage sentiment information during the summarization process. Design mechanisms to prioritize or adjust the inclusion of sentences based on their sentiment polarity to ensure that the generated summaries reflect the emotional context of the original text.

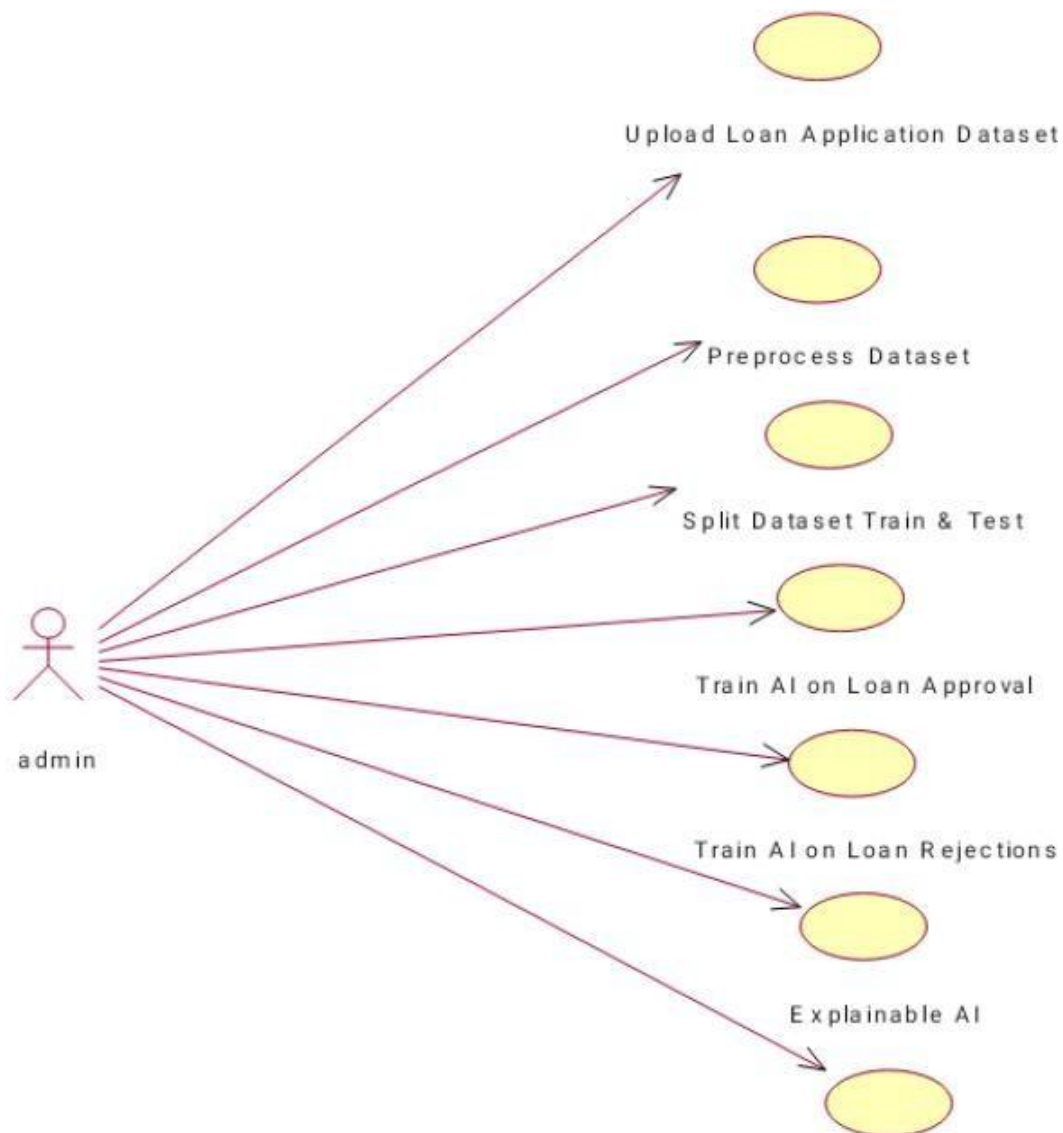
Future of Loan Approvals with Explainable AI

5. **Evaluation:** Evaluate the performance of the implemented system using standard metrics such as ROUGE (Recall-Oriented Understudy for Gisting Evaluation) for summarization quality and sentiment classification accuracy metrics for sentiment analysis. Conduct thorough evaluations using benchmark datasets to assess the effectiveness and robustness of the system.
6. **Optimization:** Optimize the system for efficiency and scalability by leveraging techniques such as parallel processing, caching, and model compression. Consider deploying the system on distributed computing frameworks or utilizing hardware accelerators (e.g., GPUs) to improve processing speed and resource utilization.
7. **User Interface:** Develop a user-friendly interface for interacting with the system, allowing users to input text and view the generated summaries along with sentiment analysis results. Design the interface to be intuitive, responsive, and accessible across different devices and platforms.
8. **Deployment:** Deploy the implemented system in production environments, considering factors such as scalability, reliability, and security. Ensure proper monitoring and maintenance procedures are in place to address potential issues and ensure continuous performance optimization.
9. **Feedback Loop:** Establish a feedback loop to gather user feedback and monitor system performance over time. Use feedback to iteratively improve the system's accuracy, usability, and effectiveness based on user requirements and evolving needs.

6. UML DIAGRAMS

6.1 USE CASE DIAGRAM:

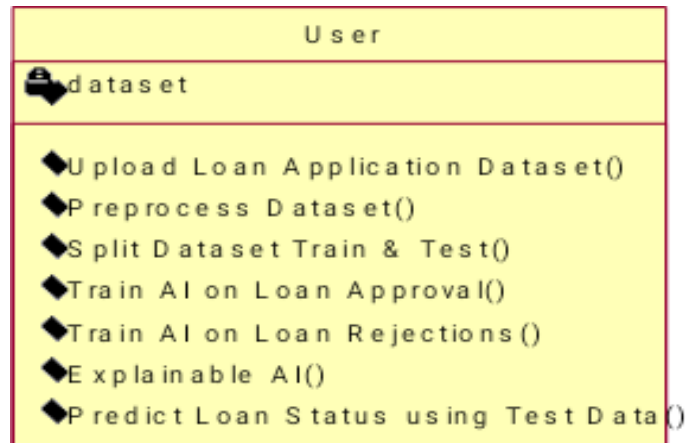
A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.



Future of Loan Approvals with Explainable AI

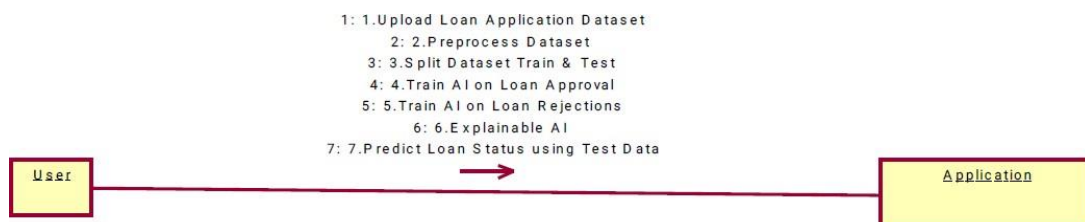
6.2 CLASS DIAGRAM:

In software engineering, a Class Diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.



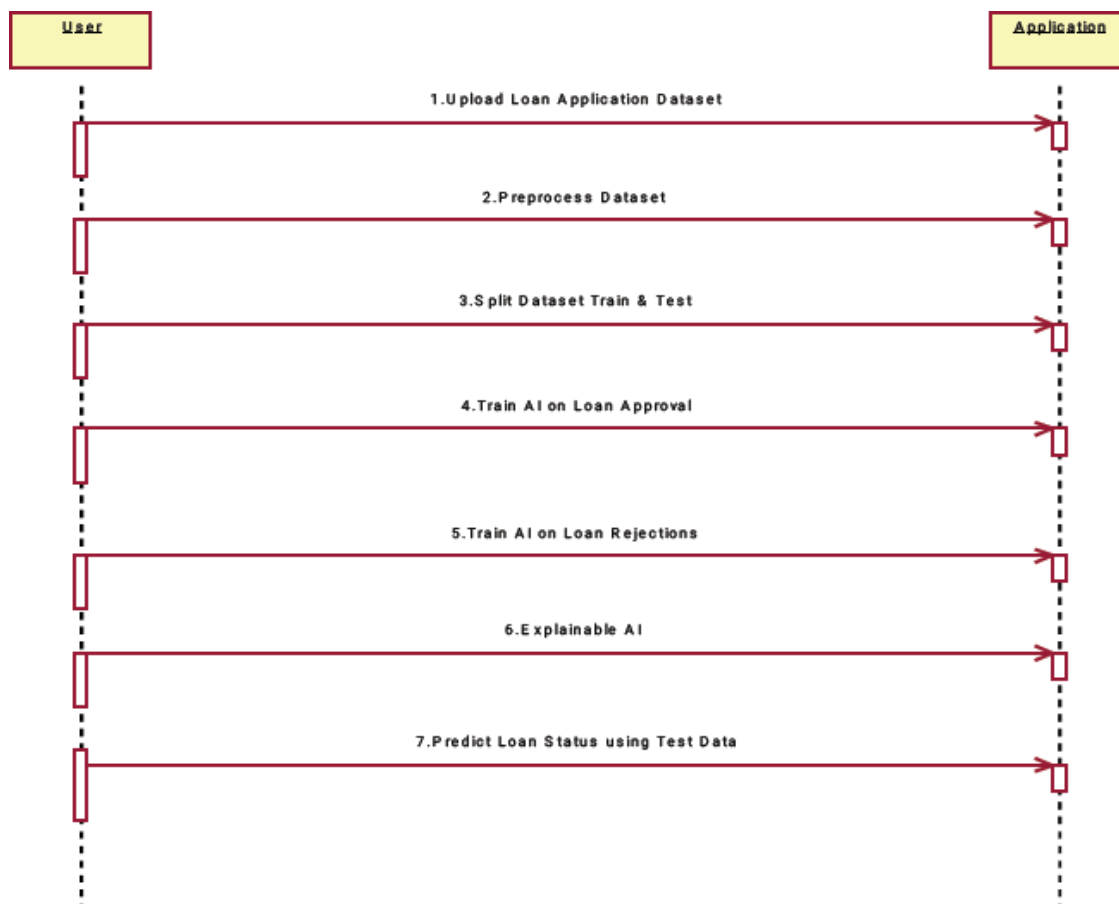
6.3 COLLABORATIVE DIAGRAM:

A Collaboration Diagram groups together the interactions between different objects. The interactions are listed as numbered interactions that help to trace the sequence of the interactions. The collaboration diagram helps to identify all the possible interactions that each object has with other objects



6.4 SEQUENCE DIAGRAM:

A Sequence Diagram represents the interaction between different objects in the system. The important aspect of a sequence diagram is that it is time-ordered. This means that the exact sequence of the interactions between the objects is represented step by step. Different objects in the sequence diagram interact with each other by passing "messages".



7. SYSTEM ENVIRONMENT

7.1 WHAT IS PYTHON?

Below are some facts about Python.

- Python is currently the most widely used multi-purpose, high-level programming language.
- Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.
- Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.
- Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc.
- The biggest strength of Python is huge collection of standard library which can be used for the following –
 - Machine Learning
 - GUI Applications (like Kivy, Tkinter, PyQt etc.)
 - Web frameworks like Django (used by YouTube, Instagram, Dropbox)
 - Image processing (like Opencv, Pillow)
 - Web scraping (like Scrapy, BeautifulSoup, Selenium)
 - Test framework
 - Multimedia

7.2 ADVANTAGES OF PYTHON

Let's see how Python dominates over other languages.

1. Extensive Libraries:

Python downloads with an extensive library and it contains code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more. So, we don't have to write the complete code for that manually.

2. Extensible:

As we have seen earlier, Python can be extended to other languages. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

3. Embeddable:

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add scripting capabilities to our code in the other language.

4. Improved Productivity:

The language's simplicity and extensive libraries render programmers more productive than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

5. IOT Opportunities:

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet of Things. This is a way to connect the language with the real world.

6. Simple and Easy:

When working with Java, you may have to create a class to print 'Hello World'. But in Python, just a print statement will do. It is also quite easy to learn, understand, and code. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

7.3 DISADVANTAGES OF PYTHON

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

1. Speed Limitations:

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in slow execution. This, however, isn't a problem unless speed is a focal point for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

2. Weak in Mobile Computing and Browsers:

While it serves as an excellent server-side language, Python is much rarely seen on the client-side. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called Carbonnelle. The reason it is not so famous despite the existence of Brython is that it isn't that secure.

3. Design Restrictions:

As you know, Python is dynamically-typed. This means that you don't need to declare the type of variable while writing the code. It uses duck-typing. But wait, what's that? Well, it just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can raise run-time errors.

4. Underdeveloped Database Access Layers:

Compared to more widely used technologies like JDBC (Java Data Base Connectivity) and ODBC (Open Data Base Connectivity), Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

5. Simple:

No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java, I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary.

HISTORY OF PYTHON

Python was developed by Guido van Rossum (a Dutch programmer) in the late 1980s and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands. Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, Smalltalk, and Unix shell and other scripting languages. Guido van Rossum wanted Python to be a high-level language that was powerful yet readable and easy to use. Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde & Informatica). The greatest achievement of ABC was to influence the design of Python. Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venner¹, Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde in Informatica (CWI).

I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it. "Later in the same Interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that possessed some of ABC's better properties, but without its problems.

So, I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers".

8. WHAT IS MACHINE LEARNING?

Before we take a look at the details of various machine learning methods, let's start by looking at what machine learning is, and what it isn't. Machine learning is often categorized as a subfield of artificial intelligence, but I find that categorization can often be misleading at first brush. The study of machine learning certainly arose from research in this context, but in the data science application of machine learning methods, it's more helpful to think of machine learning as a means of building models of data.

Fundamentally, machine learning involves building mathematical models to help understand data. "Learning" enters the fray when we give these models tunable parameters that can be adapted to observed data; in this way the program can be considered to be "learning" from the data.

Once these models have been fit to previously seen data, they can be used to predict and understand aspects of newly observed data. I'll leave to the reader the more philosophical digression regarding the extent to which this type of mathematical, model-based "learning" is similar to the "learning" exhibited by the human brain. Understanding the problem setting in machine learning is essential to using these tools effectively, and so we will start with some broad categorizations of the types of approaches we'll discuss here.

At its core, machine learning uses mathematical models to understand relationships within data. These models are trained using data so that, over time, they can make accurate predictions or decisions when given new, unseen information. This process of learning from data makes the system more flexible and adaptable compared to traditional programming, where every rule has to be defined manually.

Machine learning powers many real-world applications we interact with daily, such as facial recognition systems, virtual assistants, recommendation engines, voice-to-text converters, fraud detection tools, and more. Its ability to continuously evolve and self-improve makes it one of the most powerful tools in modern technology, especially as the amount of data in the world continues to grow.

In essence, machine learning gives computers the ability to **"learn by experience"** just like humans — using algorithms and data instead of being told exactly what to do.

Categories of Machine Learning:

At the most fundamental level, machine learning can be categorized into two main types: supervised learning and unsupervised learning.

Supervised Learning involves somehow modelling the relationship between measured features of data and some label associated with the data; once this model is determined, it can be used to apply labels to new, unknown data. This is further subdivided into classification tasks and regression tasks: in classification, the labels are discrete categories, while in regression, the labels are continuous quantities. We will see examples of both types of supervised learning in the following section.

Unsupervised Learning involves modelling the features of a dataset without reference to any label and is often described as "letting the dataset speak for itself." These models include tasks such as clustering and dimensionality reduction. Clustering algorithms identify distinct groups of data, while dimensional reduction algorithms search for more succinct representations of the data. We will see examples of both types of unsupervised learning in the following section.

Need of Machine Learning:

Human beings, at this moment, are the most intelligent and advanced species on earth because they can think, evaluate and solve complex problems. On the other side, AI is still in its initial stage and haven't surpassed human intelligence in many aspects. Then the question is that what is the need to make machine learn? The most suitable reason for doing this is, "to make decisions, based on data, with efficiency and scale".

Lately, organizations are investing heavily in newer technologies like Artificial Intelligence, Machine Learning and Deep Learning to get the key information from data to perform several real-world tasks and solve problems. We can call it data-driven decisions taken by machines, particularly to automate the process. These data-driven decisions can be used, instead of using programming logic, in the problems that cannot be programmed inherently.

8.1 CHALLENGES IN MACHINE LEARNING

While Machine Learning is rapidly evolving, making significant strides with cybersecurity and autonomous cars, this segment of AI as whole still has a long way to go. The reason behind is that ML has not been able to overcome number of challenges. The challenges that ML is facing currently are –

Quality of data – Having good-quality data for ML algorithms is one of the biggest challenges. Use of low-quality data leads to the problems related to data preprocessing and feature extraction.

Time-Consuming task – Another challenge faced by ML models is the consumption of time specially for data acquisition, feature extraction and retrieval.

Lack of specialist persons – As ML technology is still in its infancy stage, availability of expert resources is a tough job.

No clear objective for formulating business problems – Having no clear objective and well defined goal for business problems is another key challenge for ML because this technology is not that mature yet.

Issue of overfitting & underfitting – If the model is overfitting or underfitting, it cannot be represented well for the problem.

8.2 APPLICATIONS IN MACHINE LEARNING

Machine Learning is the most rapidly growing technology and according to researchers we are in the golden year of AI and ML. It is used to solve many real-world complex problems which cannot be solved with traditional approach. Following are some real-world applications of ML-

- Emotion analysis
- Sentiment analysis
- Error detection and prevention
- Weather forecasting and prediction
- Stock market analysis and forecasting

8.3 ADVANTAGES IN MACHINE LEARNING

1. Easily identifies trends and patterns:

Machine Learning can review large volumes of data and discover specific trends and patterns that would not be apparent to humans. For instance, for an e-commerce website like Amazon, it serves to understand the browsing behaviour and purchase histories of its users to help cater to the right products, deals, and reminders relevant to them. It uses the results to reveal relevant advertisements to them.

2. No human intervention needed (automation):

With ML, you don't need to babysit your project every step of the way. Since it means giving machines the ability to learn, it lets them make predictions and also improve the algorithms on their own. A common example of this is anti-virus software's; they learn to filter new threats as they are recognized. ML is also good at recognizing spam.

3. Continuous Improvement:

As ML algorithms gain experience, they keep improving in accuracy and efficiency. This lets them make better decisions. Say you need to make a weather forecast model. As the amount of data you have keeps growing, your algorithms learn to make more accurate predictions faster.

4. Handling multi-dimensional and multi-variety data:

Machine Learning algorithms are good at handling data that are multi-dimensional and multi-variety, and they can do this in dynamic or uncertain environments.

5. Wide Applications:

You could be an e-tailer or a healthcare provider and make ML work for you. Where it does apply, it holds the capability to help deliver a much more personal experience to customers while also targeting the right customers.

6. Automation and Efficiency:

Machine learning can analyze large volumes of data and make predictions or decisions automatically, reducing manual effort and improving efficiency.

8.4 DISADVANTAGES OF MACHINE LEARNING

1. Data Acquisition:

Machine Learning requires massive data sets to train on, and these should be inclusive/unbiased, and of good quality. There can also be times where they must wait for new data to be generated.

2. Time and Resources:

ML needs enough time to let the algorithms learn and develop enough to fulfill their purpose with a considerable amount of accuracy and relevancy. It also needs massive resources to function. This can mean additional requirements of computer power for you.

3. Interpretation of Results:

Another major challenge is the ability to accurately interpret results generated by the algorithms. You must also carefully choose the algorithms for your purpose.

4. High error-susceptibility:

Machine Learning is autonomous but highly susceptible to errors. Suppose you train an algorithm with data sets small enough to not be inclusive. You end up with biased predictions coming from a biased training set. This leads to irrelevant advertisements being displayed to customers. In the case of ML, such blunders can set off a chain of errors that can go undetected for long periods of time. And when they do get noticed, it takes quite some time to recognize the source of the issue, and even longer to correct it.

9. MODULES USED IN PROJECT

TensorFlow:

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library and is also used for machine learning applications such as neural networks. It is used for both research and production at Google. TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

NumPy:

NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, NumPy can also be used as an efficient multi-dimensional container of generic data. Arbitrary datatypes can be defined using NumPy which allows NumPy to seamlessly and speedily integrate with a wide variety of databases.

Pandas:

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Future of Loan Approvals with Explainable AI

Matplotlib:

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the IPython and Python shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery. For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object-oriented interface or via a set of functions familiar to MATLAB users.

Scikit-learn:

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use. Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviours.

10. HOW TO INSTALL PYTHON ON WINDOWS AND MAC?

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python, but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices.

Before you start with the installation process of Python. First, you need to know about your System Requirements. Based on your system type i.e. operating system and based processor, you must download the python version. My system type is a Windows 64-bit operating system. So, the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. Download the Python Cheat sheet here. The steps on how to install Python on Windows 10, 8 and 7 are divided into 4 parts to help understand better.

Download the Correct version into the system

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: <https://www.python.org>

Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.



Future of Loan Approvals with Explainable AI

Step 3: You can either select the Download Python for windows 3.7.4 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.7.4

Looking for a specific release?

Python releases by version number:

Release version	Release date		Click for more
Python 3.7.4	July 9, 2019	Download	Release Notes
Python 3.6.9	July 2, 2019	Download	Release Notes
Python 3.7.3	March 25, 2019	Download	Release Notes
Python 3.4.10	March 18, 2019	Download	Release Notes
Python 3.5.7	March 18, 2019	Download	Release Notes
Python 2.7.16	March 4, 2019	Download	Release Notes
Python 3.7.2	Dec. 24, 2018	Download	Release Notes

Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system.

Files

Version	Operating System	Description	MD5 Sum	File Size	GPG
Cropped source tarball	Source release		68111471e5b2db4aef7b5ab01b0f9be	23017663	S/G
XZ compressed source tarball	Source release		d33e4aa6097051c2eca45ee3604803	17133432	S/G
macOS 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later	6+28b4fa75e3daf1a442cbadce0be6	34898416	S/G
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later	5dd605c38217a45773b5e4a936b241f	28082845	S/G
Windows help file	Windows		d63999573a2c06b2ac56cad6b4f7cd2	8111761	S/G
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64T/x64	9b00c3cf8d9ec0b9abe83384a40729a2	7504391	S/G
Windows x86-64 executable installer	Windows	for AMD64/EM64T/x64	a701b4b0ad76d6bdc3043a583e563400	26480368	S/G
Windows x86-64 web-based installer	Windows	for AMD64/EM64T/x64	28c51c608b6d73aeb531a3bd351b4bd2	1362904	S/G
Windows x86 embeddable zip file	Windows		9fab3b8178841879fda94133574139d8	6741626	S/G
Windows x86 executable installer	Windows		33cc602942a54446a3d6451476394789	25663848	S/G
Windows x86 web-based installer	Windows		1b670cfa5d117d92c30963ea371d87c	1324608	S/G

- To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web based installer.
- To download Windows 64-bit python, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer.

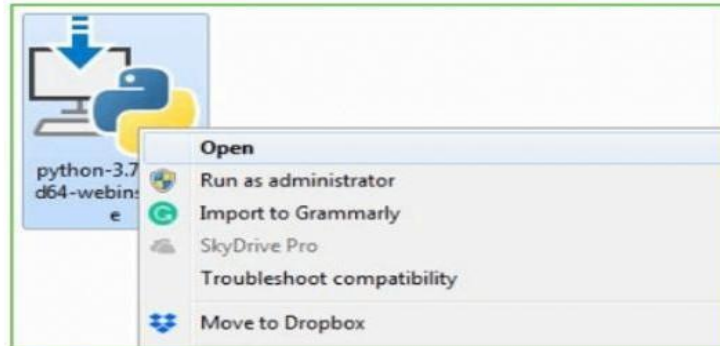
Future of Loan Approvals with Explainable AI

Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation

Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

INSTALLATION OF PYTHON

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.



Step 2: Before you click on Install Now, Make sure to put a tick on Add Python 3.7 to PATH



Step 3: Click on Install NOW After the installation is successful. Click on Close



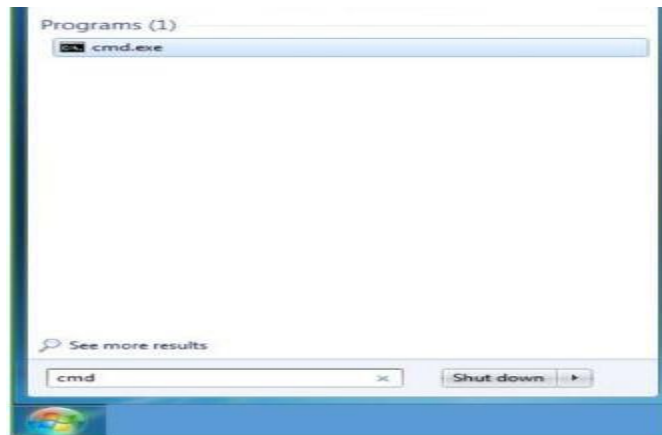
With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes

VERIFY THE PYTHON INSTALLATION

Step 1: Click on Start

Step 2: In the Windows Run Command, type “cmd”.



Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type **python -V** and press Enter

```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.
C:\Users\DELL>python -V
Python 3.7.4
C:\Users\DELL>_
```

Step 5: You will get the answer as 3.7.4

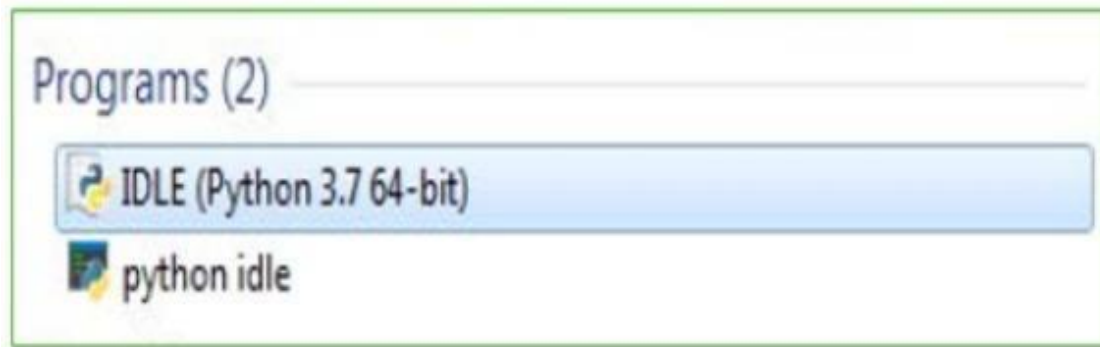
Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Future of Loan Approvals with Explainable AI

CHECK HOW THE PYTHON IDLE WORKS

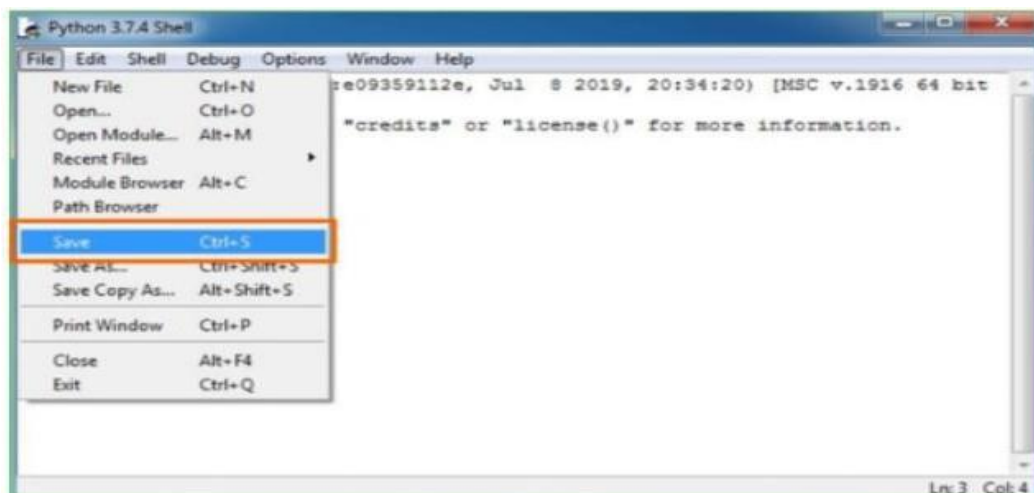
Step 1: Click on Start

Step 2: In the Windows Run command, type “python idle”.



Step 3: Click on IDLE (Python 3.7 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file. **Click on File > Click on Save**



Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

Step 6: Now for e.g. **enter**

11. SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of tests. Each test type addresses a specific testing requirement.

11.1 TYPES OF TESTS

1. UNIT TESTING:

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

2. INTEGRATION TESTING:

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

3. FUNCTIONAL TESTING:

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Future of Loan Approvals with Explainable AI

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.

4. SYSTEM TESTING:

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

5. WHITE BOX TESTING:

White Box Testing is a testing in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is used to test areas that cannot be reached from a black box level.

6. BLACK BOX TESTING:

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box. You cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

11.2 TEST STRATEGY AND APPROACH

Field testing will be performed manually, and functional tests will be written in detail.

Test objectives:

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Future of Loan Approvals with Explainable AI

Features to be tested:

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page

INTEGRATION TESTING:

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

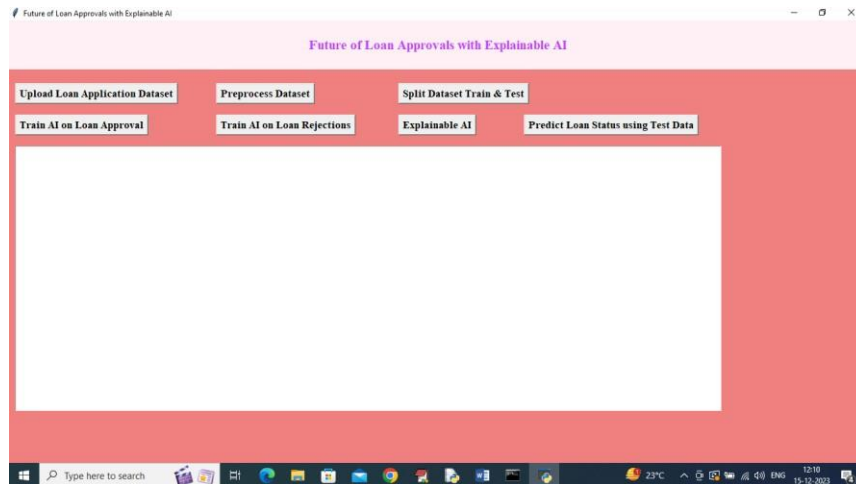
ACCEPTANCE TESTING:

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

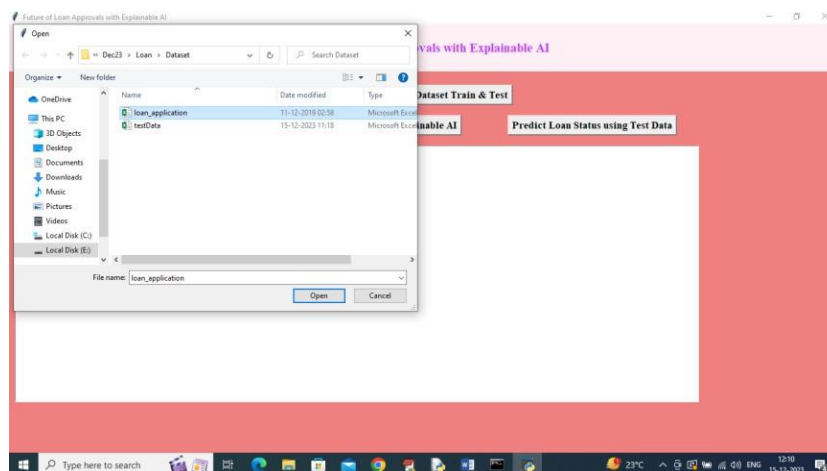
Test Results: All the test cases mentioned above passed successfully.

12. SCREENSHOTS

To run project double click on 'run.bat' file to get below screen

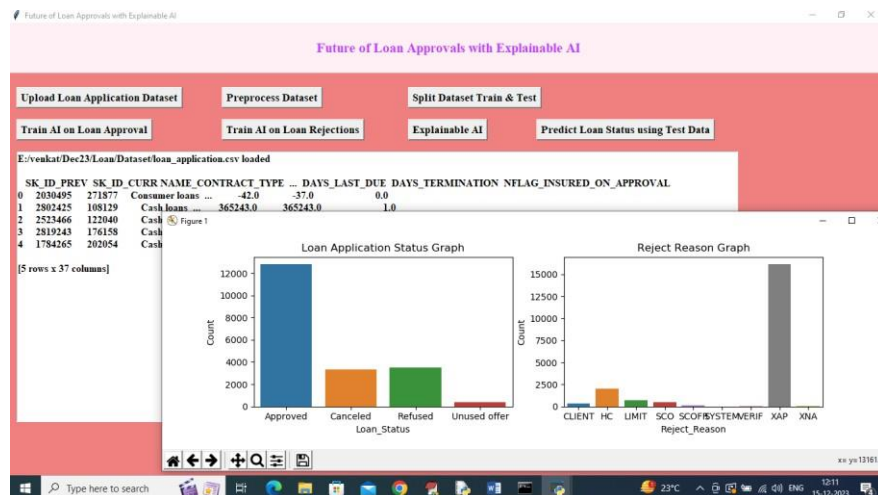


In above screen click on 'Upload Loan Application Dataset' button to upload dataset and then will get below output.

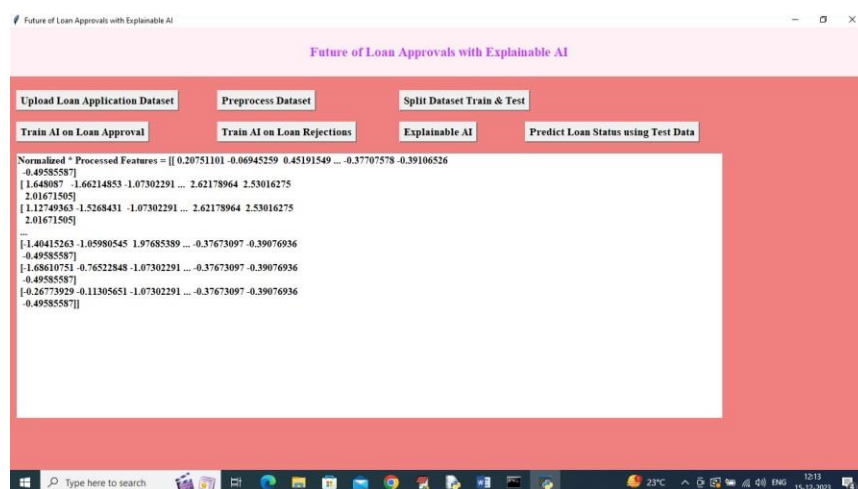


In above screen selecting and uploading 'loan_application.csv' file and then click on 'Open' button to load dataset and get below output.

Future of Loan Approvals with Explainable AI

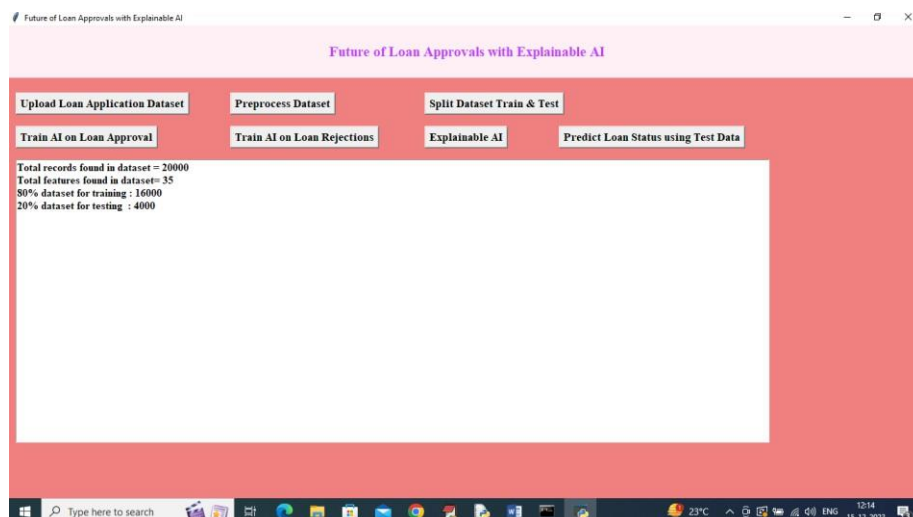


In above screen dataset loaded and in text area can see few records from dataset and in first graph x-axis represents LOAN STATUS and y-axis represents Number of Records available in that LOAN STATUS class label. In second graph x-axis represents REJECTION REASON and y-axis represents records size and in dataset we have both numeric and nonnumeric values so to convert to numeric data then click on 'Pre-process Dataset' button to get below output.

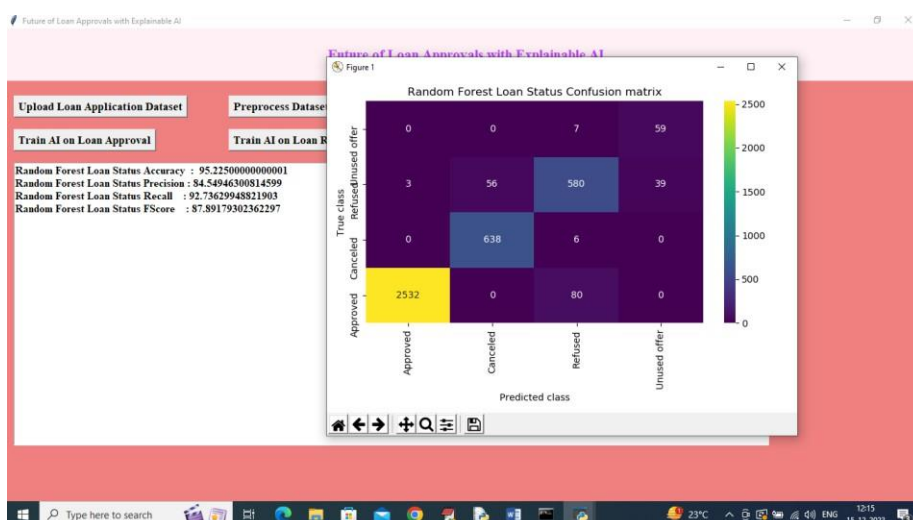


In above screen dataset converted to numeric format and then click on 'Split Dataset Train & Test' button to split dataset into train and test and then will get below output.

Future of Loan Approvals with Explainable AI

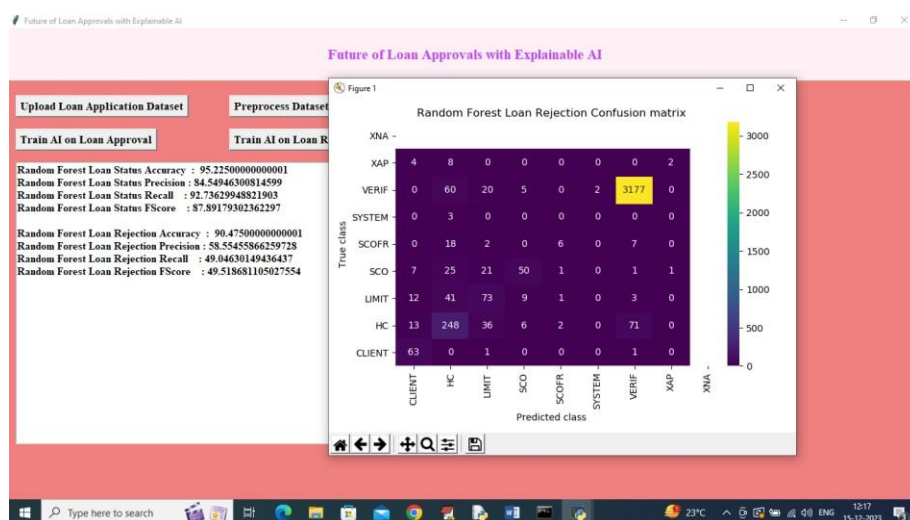


In above screen can see dataset size with total number of features and then can see TRAIN and TEST size and now click on 'Train AO on Loan Status Approval' button to train AI and get below output.

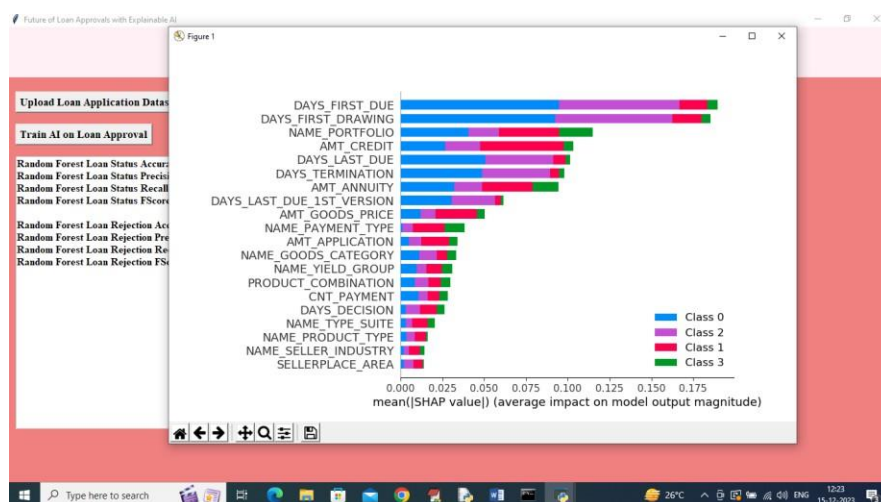


In above screen AI Random Forest got 95% accuracy on Loan STATUS and can see other metrics also. In above confusion matrix graph x-axis represents 'LOAN STATUS Predicted Labels' and y-axis represents TRUE labels and all boxes in diagonal contains correct prediction count and remaining blue boxes contains incorrect prediction count which are very few. Now click on 'Train AI on Loan Rejections' button to train AI on rejection reason and get below output.

Future of Loan Approvals with Explainable AI

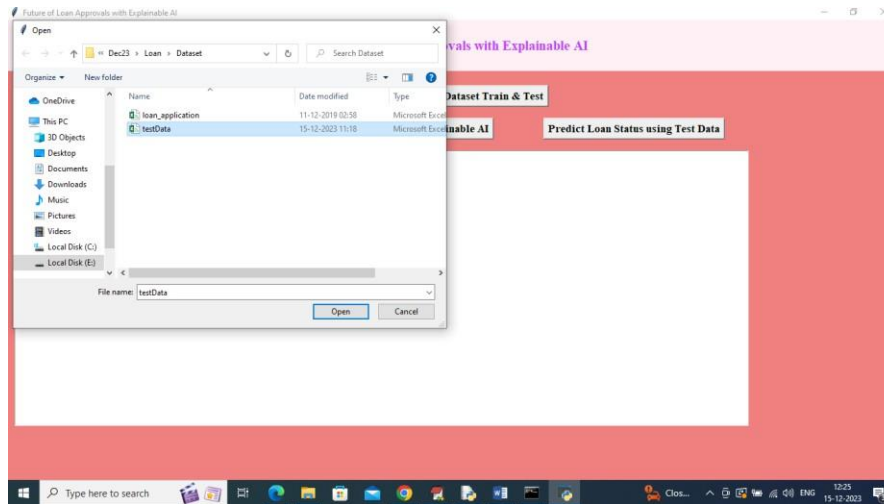


In above screen AI on REJECTION got 90% accuracy and in confusion matrix graph x-axis represents 'Rejection Reason Predicted Labels' and y-axis represents True label and in diagonal boxes we can see correct prediction count and remaining boxes contains incorrect prediction count. Now click on 'Explainable AI' button to get below features explanation on prediction.

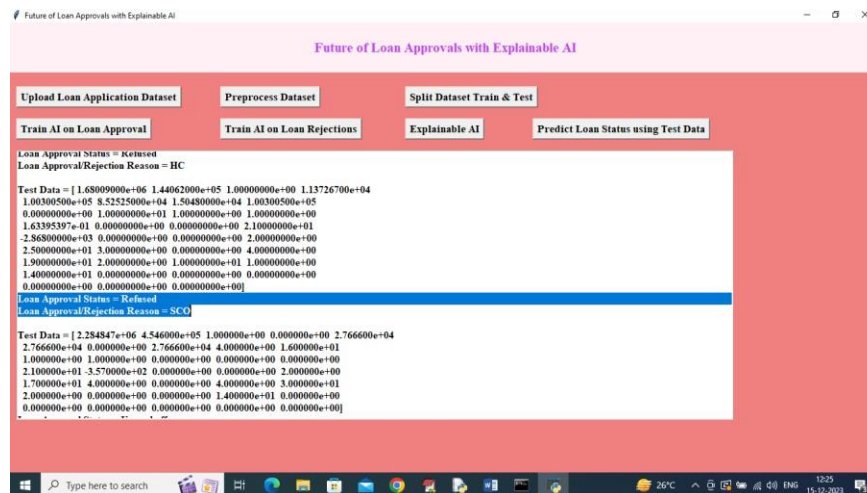


In above SHAP explanation screen in each bar we can see 4 different colors and each color represents one class label and based on color percentage we can say which feature names is contributing how much to predict that class label. Now close above graph and then click on 'Predict Loan Status using Test Data' button to upload test data and then will get below prediction.

Future of Loan Approvals with Explainable AI

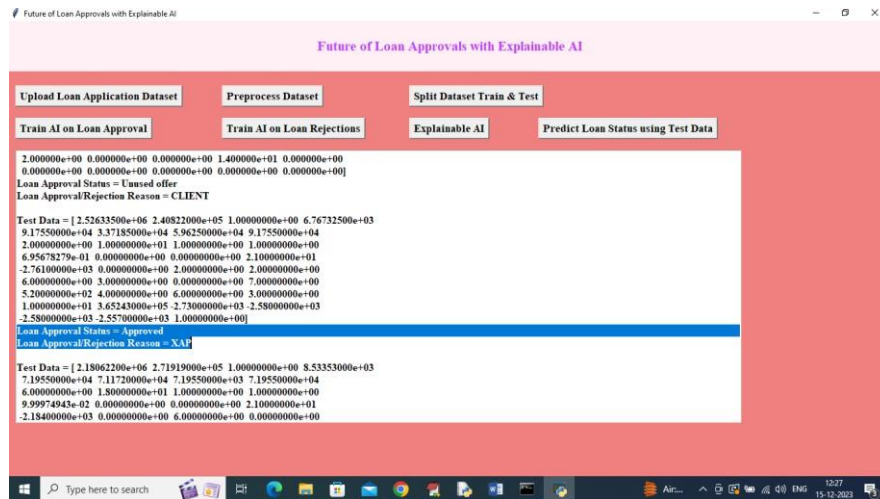


In above screen selecting and uploading testData.csv file and then click on ‘Open’ button to get below output.

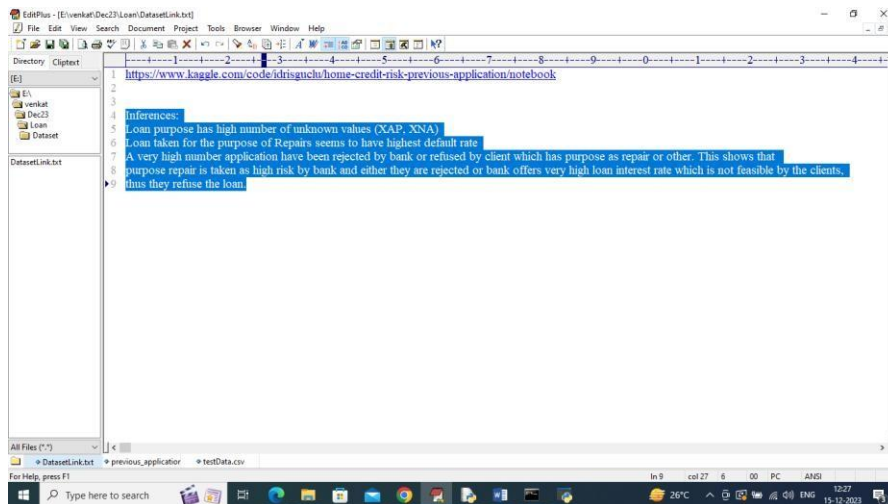


In above screen in square bracket we can see test data and then in blue color selected line next to TEST data we can see LOAN STATUS prediction and REASON details. Scroll down above output to view all predictions.

Future of Loan Approvals with Explainable AI



In above screen can see other prediction output. For Reason Rejected code you can read below description.



In above screen read blue color selected text to know about REJECTED REASON codes.

13. SOURCE CODE

```
from tkinter import *
import tkinter
from tkinter import
filedialog
import numpy as np
from tkinter import
simpdialog
from
sklearn.model_selection import train_test_split
from tkinter import ttk
from tkinter.filedialog
import askopenfilename
import os
import pickle
import matplotlib.pyplot
as plt
import seaborn as sns
from sklearn.metrics
import accuracy_score
import pandas as pd
from sklearn.metrics
import precision_score
from sklearn.metrics
import recall_score
from sklearn.metrics
import f1_score
from sklearn.metrics
import confusion_matrix
from
```

Future of Loan Approvals with Explainable AI

```
sklearn.preprocessing import StandardScaler
from
sklearn.preprocessing import LabelEncoder
from sklearn.ensemble
import RandomForestClassifier#importing ML classes
import shap

main = tkinter.Tk()
main.title("Future
of Loan Approvals with Explainable AI") #designing main screen
main.geometry("1000x650")

global filename,
loan_status, loan_reject_reason, status_names, reject_names, label_encoder,
scaler, cols
global loan_X_train,
loan_X_test, loan_y_train, loan_y_test
global reject_X_train,
reject_X_test, reject_y_train, reject_y_test
global accuracy,
precision, recall, fscore
global rf, reject_rf,
dataset, X

def loadDataset():
    global filename, dataset, loan_status,
    loan_reject_reason, status_names, reject_names
```

Future of Loan Approvals with Explainable AI

```
filename =
filedialog.askopenfilename(initialdir="Dataset")
    text.delete('1.0', END)
    text.insert(END,filename+"
loaded\n\n")
    dataset = pd.read_csv(filename,
nrows=20000)
    text.insert(END,str(dataset.head()))
    loan_status =
dataset['NAME_CONTRACT_STATUS']
    loan_reject_reason =
dataset['CODE_REJECT_REASON']
    #visualizing loan status class labels
    status_names, status_count =
np.unique(loan_status, return_counts = True)
    reject_names, reject_count =
np.unique(loan_reject_reason, return_counts = True)
    loan_df = []
    for i in range(len(status_names)):
        loan_df.append([status_names[i],
status_count[i]])
    loan_df = pd.DataFrame(loan_df,
columns=['Loan_Status', 'Count'])
    reject_df = []
    for i in range(len(reject_names)):
        reject_df.append([reject_names[i],
reject_count[i]])
    reject_df = pd.DataFrame(reject_df,
columns=['Reject_Reason', 'Count'])
```


Future of Loan Approvals with Explainable AI

```
fig, axs = plt.subplots(1, 2, figsize=(14,
4))

sns.barplot(x="Loan_Status",
y='Count', data=loan_df, ax=axs[0])

sns.barplot(x="Reject_Reason",
y='Count', data=reject_df, ax=axs[1])

axs[0].set_title("Loan Application
Status Graph")

axs[1].set_title("Reject Reason
Graph")

plt.show()
```

```
def processDataset():
    text.delete('1.0', END)
    global dataset, scaler, label_encoder,
cols, loan_status, loan_reject_reason, X
    label_encoder = []
    #converting non-numeric data to numeric
values
    dataset.fillna(0, inplace = True)
    label_encoder = []
    columns = dataset.columns
    types = dataset.dtypes.values
    cols = []
    for i in range(len(types)):
        name = types[i]
        if name == 'object': #finding column
with object type
            le = LabelEncoder()
```

Future of Loan Approvals with Explainable AI

```
dataset[columns[i]] =
pd.Series(le.fit_transform(dataset[columns[i]].astype(str)))#encode all str
columns to numeric

    label_encoder.append(le)
    cols.append(columns[i])

dataset.fillna(0, inplace = True)

loan_status =
dataset['NAME_CONTRACT_STATUS']

    loan_reject_reason =
dataset['CODE_REJECT_REASON']

    dataset.drop(['NAME_CONTRACT_STATUS',
'CODE_REJECT_REASON'], axis = 1,inplace=True)

X = dataset.values

scaler = StandardScaler()

X = scaler.fit_transform(X)#normalize train
features

    text.insert(END,"Normalized *
Processed Features = "+str(X))

def splitDataset():
    text.delete('1.0', END)

    global X, loan_status, loan_reject_reason

    global loan_X_train, loan_X_test,
loan_y_train, loan_y_test

    global reject_X_train, reject_X_test,
reject_y_train, reject_y_test

    #split dataset into train and test

    loan_X_train, loan_X_test, loan_y_train,
loan_y_test = train_test_split(X, loan_status, test_size = 0.2)

    reject_X_train, reject_X_test,
```

Future of Loan Approvals with Explainable AI

```
reject_y_train, reject_y_test = train_test_split(X, loan_reject_reason,  
test_size = 0.2)
```

```
    text.insert(END,"Total records found  
in dataset = "+str(X.shape[0])+"\n")  
    text.insert(END,"Total features found  
in dataset= "+str(X.shape[1])+"\n")  
    text.insert(END,"80% dataset for  
training : "+str(loan_X_train.shape[0])+"\n")  
    text.insert(END,"20% dataset for  
testing :  
"+str(loan_X_test.shape[0])+"\n")
```

```
def
```

```
calculateMetrics(algorithm, predict, y_test, label_names):
```

```
    a = accuracy_score(y_test,predict)*100  
    p = precision_score(y_test,  
predict,average='macro') * 100  
    r = recall_score(y_test,  
predict,average='macro') * 100  
    f = f1_score(y_test,  
predict,average='macro') * 100  
    accuracy.append(a)  
    precision.append(p)  
    recall.append(r)  
    fscore.append(f)  
    text.insert(END,algorithm+"  
Accuracy : "+str(a)+"\n")  
    text.insert(END,algorithm+" Precision  
: "+str(p)+"\n")  
    text.insert(END,algorithm+"
```

Future of Loan Approvals with Explainable AI

```
Recall    : "+str(r)+"\n")
    text.insert(END,algorithm+"
FScore    :
"+str(f)+"\n\n")
    conf_matrix = confusion_matrix(y_test,
predict)
    plt.figure(figsize =(7, 5))
    ax = sns.heatmap(conf_matrix, xticklabels =
label_names, yticklabels = label_names, annot = True, cmap="viridis"
,fmt ="g");
    ax.set_ylim([0,len(label_names)])
    plt.title(algorithm+" Confusion
matrix")
    plt.xticks(rotation=90)
    plt.ylabel('True class')
    plt.xlabel('Predicted class')
    plt.tight_layout()
    plt.show()

def aiApproval():
    text.delete('1.0', END)
    global loan_X_train, loan_X_test,
loan_y_train, loan_y_test
    global reject_X_train, reject_X_test,
reject_y_train, reject_y_test
    global accuracy, precision, recall, fscore,
rf, status_names
    #define global variables to save accuracy
and other metrics
    accuracy = []
```

Future of Loan Approvals with Explainable AI

```
precision = []
recall = []
fscore = []

rf = RandomForestClassifier()
rf.fit(loan_X_train, loan_y_train)
predict = rf.predict(loan_X_test)
calculateMetrics("Random Forest Loan
Status", predict, loan_y_test, status_names)

def aiReject():
    global reject_X_train, reject_X_test,
reject_y_train, reject_y_test
    global accuracy, precision, recall, fscore,
rf, reject_names, reject_rf
    reject_rf = RandomForestClassifier()
    reject_rf.fit(reject_X_train,
reject_y_train)
    predict = reject_rf.predict(reject_X_test)
    calculateMetrics("Random Forest Loan
Rejection", predict, reject_y_test, reject_names)

def explainAI():
    global rf
    features_names = ['SK_ID_PREV',
'SK_ID_CURR', 'NAME_CONTRACT_TYPE', 'AMT_ANNUITY', 'AMT_APPLICATION',
'AMT_CREDIT', 'AMT_DOWN_PAYMENT', 'AMT_GOODS_PRICE',
'WEEKDAY_APPR_PROCESS_START', 'HOUR_APPR_PROCESS_START',
'FLAG_LAST_APPL_PER_CONTRACT', 'NFLAG_LAST_APPL_IN_DAY',
'RATE_DOWN_PAYMENT',
'RATE_INTEREST_PRIMARY',
```

Future of Loan Approvals with Explainable AI

```
'RATE_INTEREST_PRIVILEGED', 'NAME_CASH_LOAN_PURPOSE', 'DAYS_DECISION',  
    'NAME_PAYMENT_TYPE',  
'NAME_TYPE_SUITE', 'NAME_CLIENT_TYPE', 'NAME_GOODS_CATEGORY',  
'NAME_PORTFOLIO',  
    'NAME_PRODUCT_TYPE',  
'CHANNEL_TYPE', 'SELLERPLACE_AREA', 'NAME_SELLER_INDUSTRY',  
'CNT_PAYMENT',  
'NAME_YIELD_GROUP',  
    'PRODUCT_COMBINATION',  
'DAYS_FIRST_DRAWING', 'DAYS_FIRST_DUE', 'DAYS_LAST_DUE_1ST_VERSION',  
'DAYS_LAST_DUE', 'DAYS_TERMINATION',  
    'NFLAG_INSURED_ON_APPROVAL']
```

```
explainer = shap.TreeExplainer(rf)  
shap_values =  
explainer.shap_values(loan_X_test[0:200])  
shap.summary_plot(shap_values,  
feature_names=features_names)  
plt.show()
```

```
def predict():  
    text.delete('1.0', END)  
    global rf, reject_rf, scaler,  
label_encoder, cols, status_names, reject_names  
    filename =  
filedialog.askopenfilename(initialdir="Dataset")  
    test_data = pd.read_csv(filename)  
    test_data.fillna(0, inplace = True)  
    for i in range(len(cols)):  
        test_data[cols[i]] =
```

Future of Loan Approvals with Explainable AI

```
pd.Series(label_encoder[i].transform(test_data[cols[i]].astype(str)))

test_data.fillna(0, inplace = True)

test_data.drop(['NAME_CONTRACT_STATUS',
'CODE_REJECT_REASON'], axis = 1,inplace=True)

test = test_data.values

X = test_data.values

X = scaler.transform(X)

loan_predict = rf.predict(X)

reject_predict = reject_rf.predict(X)

for i in range(len(loan_predict)):

    text.insert(END,"Test Data =

"+str(test[i])+"\n")

    text.insert(END,"Loan Approval

Status = "+str(status_names[loan_predict[i]])+"\n")

    text.insert(END,"Loan

Approval/Rejection Reason =

"+str(reject_names[reject_predict[i]])+"\n\n")
```

```
font = ('times', 16,

'bold')

title = Label(main,

text='Future of Loan Approvals with Explainable AI', justify=LEFT)

title.config(bg='lavender

blush', fg='DarkOrchid1')

title.config(font=font)

title.config(height=3,

width=120)

title.place(x=100,y=5)

title.pack()
```

Future of Loan Approvals with Explainable AI

```
font1 = ('times', 13,  
'bold')
```

```
uploadButton =  
Button(main, text="Upload Loan Application Dataset",  
command=loadDataset)  
uploadButton.place(x=10,y=100)  
uploadButton.config(font=font1)
```

```
processButton =  
Button(main, text="Preprocess Dataset", command=processDataset)  
processButton.place(x=330,y=100)  
processButton.config(font=font1)
```

```
splitButton =  
Button(main, text="Split Dataset Train & Test",  
command=splitDataset)  
splitButton.place(x=620,y=100)  
splitButton.config(font=font1)
```

```
approvalButton =  
Button(main, text="Train AI on Loan Approval", command=aiApproval)  
approvalButton.place(x=10,y=150)  
approvalButton.config(font=font1)
```

```
rejectButton =  
Button(main, text="Train AI on Loan Rejections", command=aiReject)  
rejectButton.place(x=330,y=150)  
rejectButton.config(font=font1)
```


Future of Loan Approvals with Explainable AI

```
explainButton =  
Button(main, text="Explainable AI", command=explainAI)  
explainButton.place(x=620,y=150)  
explainButton.config(font=font1)
```

```
predictButton =  
Button(main, text="Predict Loan Status using Test Data",  
command=predict)  
predictButton.place(x=820,y=150)  
predictButton.config(font=font1)
```

```
font1 = ('times', 12,  
'bold')  
text=Text(main,height=22,width=140)  
scroll=Scrollbar(text)  
text.configure(yscrollcommand=scroll.set)  
text.place(x=10,y=200)  
text.config(font=font1)
```

```
main.config(bg='light  
coral')  
main.mainloop()
```

```
from tkinter import *  
import tkinter  
from tkinter import  
filedialog  
import numpy as np  
from tkinter import  
simplifiedialog
```

Future of Loan Approvals with Explainable AI

```
from
sklearn.model_selection import train_test_split
from tkinter import ttk
from tkinter.filedialog
import askopenfilename
import os
import pickle
import matplotlib.pyplot
as plt
import seaborn as sns
from sklearn.metrics
import accuracy_score
import pandas as pd
from sklearn.metrics
import precision_score
from sklearn.metrics
import recall_score
from sklearn.metrics
import f1_score
from sklearn.metrics
import confusion_matrix
from
sklearn.preprocessing import StandardScaler
from
sklearn.preprocessing import LabelEncoder
from sklearn.ensemble
import RandomForestClassifier#importing ML classes
import shap
```

Future of Loan Approvals with Explainable AI

```
main = tkinter.Tk()
main.title("Future
of Loan Approvals with Explainable AI") #designing main screen
main.geometry("1000x650")

global filename,
loan_status, loan_reject_reason, status_names, reject_names, label_encoder,
scaler, cols
global loan_X_train,
loan_X_test, loan_y_train, loan_y_test
global reject_X_train,
reject_X_test, reject_y_train, reject_y_test
global accuracy,
precision, recall, fscore
global rf, reject_rf,
dataset, X

def loadDataset():
    global filename, dataset, loan_status,
    loan_reject_reason, status_names, reject_names
    filename =
filedialog.askopenfilename(initialdir="Dataset")
    text.delete('1.0', END)
    text.insert(END,filename+"
loaded\n\n")
    dataset = pd.read_csv(filename,
nrows=20000)
    text.insert(END,str(dataset.head()))
    loan_status =
```

Future of Loan Approvals with Explainable AI

```
dataset['NAME_CONTRACT_STATUS']

loan_reject_reason =
dataset['CODE_REJECT_REASON']

#visualizing loan status class labels

status_names, status_count =
np.unique(loan_status, return_counts = True)

reject_names, reject_count =
np.unique(loan_reject_reason, return_counts = True)

loan_df = []
for i in range(len(status_names)):
    loan_df.append([status_names[i],
status_count[i]])

loan_df = pd.DataFrame(loan_df,
columns=['Loan_Status', 'Count'])

reject_df = []
for i in range(len(reject_names)):
    reject_df.append([reject_names[i],
reject_count[i]])

reject_df = pd.DataFrame(reject_df,
columns=['Reject_Reason', 'Count'])


fig, axs = plt.subplots(1, 2, figsize=(14,
4))

sns.barplot(x="Loan_Status",
y='Count', data=loan_df, ax=axs[0])

sns.barplot(x="Reject_Reason",
y='Count', data=reject_df, ax=axs[1])

axs[0].set_title("Loan Application
Status Graph")
```

Future of Loan Approvals with Explainable AI

```
axs[1].set_title("Reject Reason
Graph")
plt.show()

def processDataset():
    text.delete('1.0', END)
    global dataset, scaler, label_encoder,
cols, loan_status, loan_reject_reason, X
    label_encoder = []
    #converting non-numeric data to numeric
values
    dataset.fillna(0, inplace = True)
    label_encoder = []
    columns = dataset.columns
    types = dataset.dtypes.values
    cols = []
    for i in range(len(types)):
        name = types[i]
        if name == 'object': #finding column
with object type
            le = LabelEncoder()
            dataset[columns[i]] =
pd.Series(le.fit_transform(dataset[columns[i]].astype(str)))#encode all str
columns to numeric
            label_encoder.append(le)
            cols.append(columns[i])
            dataset.fillna(0, inplace = True)
            loan_status =
dataset['NAME_CONTRACT_STATUS']
```

Future of Loan Approvals with Explainable AI

```
loan_reject_reason =
dataset['CODE_REJECT_REASON']
dataset.drop(['NAME_CONTRACT_STATUS',
'CODE_REJECT_REASON'], axis = 1,inplace=True)
X = dataset.values
scaler = StandardScaler()
X = scaler.fit_transform(X)#normalize train
features
text.insert(END,"Normalized *
Processed Features = "+str(X))

def splitDataset():
    text.delete('1.0', END)
    global X, loan_status, loan_reject_reason
    global loan_X_train, loan_X_test,
loan_y_train, loan_y_test
    global reject_X_train, reject_X_test,
reject_y_train, reject_y_test
    #split dataset into train and test
    loan_X_train, loan_X_test, loan_y_train,
loan_y_test = train_test_split(X, loan_status, test_size = 0.2)
    reject_X_train, reject_X_test,
reject_y_train, reject_y_test = train_test_split(X, loan_reject_reason,
test_size = 0.2)
    text.insert(END,"Total records found
in dataset = "+str(X.shape[0])+"\n")
    text.insert(END,"Total features found
in dataset= "+str(X.shape[1])+"\n")
    text.insert(END,"80% dataset for
training : "+str(loan_X_train.shape[0])+"\n")
```

Future of Loan Approvals with Explainable AI

```
text.insert(END,"20% dataset for
testing :
"+str(loan_X_test.shape[0])+"\n")

def
calculateMetrics(algorithm, predict, y_test, label_names):
    a = accuracy_score(y_test,predict)*100
    p = precision_score(y_test,
predict,average='macro') * 100
    r = recall_score(y_test,
predict,average='macro') * 100
    f = f1_score(y_test,
predict,average='macro') * 100
    accuracy.append(a)
    precision.append(p)
    recall.append(r)
    fscore.append(f)
    text.insert(END,algorithm+"
Accuracy : "+str(a)+"\n")
    text.insert(END,algorithm+" Precision
: "+str(p)+"\n")
    text.insert(END,algorithm+"
Recall : "+str(r)+"\n")
    text.insert(END,algorithm+"
FScore :
"+str(f)+"\n\n")
    conf_matrix = confusion_matrix(y_test,
predict)
    plt.figure(figsize =(7, 5))
```

Future of Loan Approvals with Explainable AI

```
ax = sns.heatmap(conf_matrix, xticklabels =
label_names, yticklabels = label_names, annot = True, cmap="viridis"
,fmt = "g");
    ax.set_ylim([0,len(label_names)])
    plt.title(algorithm+" Confusion
matrix")
    plt.xticks(rotation=90)
    plt.ylabel("True class")
    plt.xlabel("Predicted class")
    plt.tight_layout()
    plt.show()

def aiApproval():
    text.delete('1.0', END)
    global loan_X_train, loan_X_test,
loan_y_train, loan_y_test
    global reject_X_train, reject_X_test,
reject_y_train, reject_y_test
    global accuracy, precision, recall, fscore,
rf, status_names
    #define global variables to save accuracy
and other metrics
    accuracy = []
    precision = []
    recall = []
    fscore = []
    rf = RandomForestClassifier()
    rf.fit(loan_X_train, loan_y_train)
    predict = rf.predict(loan_X_test)
```


Future of Loan Approvals with Explainable AI

```
calculateMetrics("Random Forest Loan  
Status", predict, loan_y_test, status_names)
```

```
def aiReject():  
    global reject_X_train, reject_X_test,  
    reject_y_train, reject_y_test  
    global accuracy, precision, recall, fscore,  
    rf, reject_names, reject_rf  
    reject_rf = RandomForestClassifier()  
    reject_rf.fit(reject_X_train,  
    reject_y_train)  
    predict = reject_rf.predict(reject_X_test)  
    calculateMetrics("Random Forest Loan  
Rejection", predict, reject_y_test, reject_names)
```

```
def explainAI():  
    global rf  
    features_names = ['SK_ID_PREV',  
    'SK_ID_CURR', 'NAME_CONTRACT_TYPE', 'AMT_ANNUITY', 'AMT_APPLICATION',  
    'AMT_CREDIT', 'AMT_DOWN_PAYMENT', 'AMT_GOODS_PRICE',  
    'WEEKDAY_APPR_PROCESS_START', 'HOUR_APPR_PROCESS_START',  
    'FLAG_LAST_APPL_PER_CONTRACT', 'NFLAG_LAST_APPL_IN_DAY',  
    'RATE_DOWN_PAYMENT',  
    'RATE_INTEREST_PRIMARY',  
    'RATE_INTEREST_PRIVILEGED', 'NAME_CASH_LOAN_PURPOSE', 'DAYS_DECISION',  
    'NAME_PAYMENT_TYPE',  
    'NAME_TYPE_SUITE', 'NAME_CLIENT_TYPE', 'NAME_GOODS_CATEGORY',  
    'NAME_PORTFOLIO',  
    'NAME_PRODUCT_TYPE',  
    'CHANNEL_TYPE', 'SELLERPLACE_AREA', 'NAME_SELLER_INDUSTRY',
```

Future of Loan Approvals with Explainable AI

```
'CNT_PAYMENT',
'NAME_YIELD_GROUP',
    'PRODUCT_COMBINATION',
'DAYS_FIRST_DRAWING', 'DAYS_FIRST_DUE', 'DAYS_LAST_DUE_1ST_VERSION',
'DAYS_LAST_DUE', 'DAYS_TERMINATION',
    'NFLAG_INSURED_ON_APPROVAL']
```

```
explainer = shap.TreeExplainer(rf)
shap_values =
explainer.shap_values(loan_X_test[0:200])
shap.summary_plot(shap_values,
feature_names=features_names)
plt.show()
```

```
def predict():
    text.delete('1.0', END)
    global rf, reject_rf, scaler,
label_encoder, cols, status_names, reject_names
    filename =
filedialog.askopenfilename(initialdir="Dataset")
    test_data = pd.read_csv(filename)
    test_data.fillna(0, inplace = True)
    for i in range(len(cols)):
        test_data[cols[i]] =
pd.Series(label_encoder[i].transform(test_data[cols[i]].astype(str)))
        test_data.fillna(0, inplace = True)
    test_data.drop(['NAME_CONTRACT_STATUS',
'CODE_REJECT_REASON'], axis = 1,inplace=True)
    test = test_data.values
    X = test_data.values
```

Future of Loan Approvals with Explainable AI

```
X = scaler.transform(X)
loan_predict = rf.predict(X)
reject_predict = reject_rf.predict(X)
for i in range(len(loan_predict)):
    text.insert(END,"Test Data =
"+str(test[i])+"\n")
    text.insert(END,"Loan Approval
Status = "+str(status_names[loan_predict[i]])+"\n")
    text.insert(END,"Loan
Approval/Rejection Reason =
"+str(reject_names[reject_predict[i]])+"\n\n")
```

```
font = ('times', 16,
'bold')
title = Label(main,
text='Future of Loan Approvals with Explainable AI', justify=LEFT)
title.config(bg='lavender
blush', fg='DarkOrchid1')
title.config(font=font)
title.config(height=3,
width=120)
title.place(x=100,y=5)
title.pack()
```

```
font1 = ('times', 13,
'bold')
uploadButton =
Button(main, text="Upload Loan Application Dataset",
command=loadDataset)
```

Future of Loan Approvals with Explainable AI

```
uploadButton.place(x=10,y=100)
```

```
uploadButton.config(font=font1)
```

```
processButton =
```

```
Button(main, text="Preprocess Dataset", command=processDataset)
```

```
processButton.place(x=330,y=100)
```

```
processButton.config(font=font1)
```

```
splitButton =
```

```
Button(main, text="Split Dataset Train & Test",
```

```
command=splitDataset)
```

```
splitButton.place(x=620,y=100)
```

```
splitButton.config(font=font1)
```

```
approvalButton =
```

```
Button(main, text="Train AI on Loan Approval", command=aiApproval)
```

```
approvalButton.place(x=10,y=150)
```

```
approvalButton.config(font=font1)
```

```
rejectButton =
```

```
Button(main, text="Train AI on Loan Rejections", command=aiReject)
```

```
rejectButton.place(x=330,y=150)
```

```
rejectButton.config(font=font1)
```

```
explainButton =
```

```
Button(main, text="Explainable AI", command=explainAI)
```

```
explainButton.place(x=620,y=150)
```

```
explainButton.config(font=font1)
```

```
predictButton =
```

Future of Loan Approvals with Explainable AI

```
Button(main, text="Predict Loan Status using Test Data",
command=predict)
predictButton.place(x=820,y=150)
predictButton.config(font=font1)

font1 = ('times', 12,
'bold')
text=Text(main,height=22,width=140)
scroll=Scrollbar(text)
text.configure(yscrollcommand=scroll.set)
text.place(x=10,y=200)
text.config(font=font1)

main.config(bg='light
coral')
main.mainloop()
```

14. CONCLUSION

In this paper, we presented the methodology to develop the belief-rule-based (BRB) system as an explainable AI decision-support-system to automate the underwriting process of lend loans. Unlike blackbox models, the BRB system can explicitly accommodate expert knowledge and can also learn from data by supervised learning, though the acquisition of expert knowledge can be a time-consuming and labourintensive task. The decision-making process in this system can be explained by the importance of rules activated by a data point representing a loan application and by the contribution of attributes in activated rules. Through a business case study, we have demonstrated that the proposed AI decision-support-system provides a good trade-off between prediction accuracy and explainability. The importance of activated rules and their attributes in the rules help to understand the reasoning behind the decisions. The textual explanations initiated by the chain of events in the factual-rule-base to the heuristic-rule-base could be sent to rejected applicants as reasons for denying their loan applications. The BRB structure and its decision-making process can easily be narrated and understood by non-technical people. A transparent decision-making process provides sufficient assurance to executives to allow the deployment of this system in their companies.

15. REFERENCE

1. **Abdou, H. A. (2009).** Genetic programming for credit scoring: The case of Egyptian public sector banks. *Expert systems with applications*, 36(9), 11402-11417.
2. **Abellán, J., & Castellano, J. G. (2017).** A comparative study on base classifiers in ensemble methods for credit scoring. *Expert Systems with Applications*, 73, 1-10.
3. **Adadi, A., & Berrada, M. (2018).** Peeking inside the black-box: A survey on Explainable Artificial Intelligence (XAI). *IEEE Access*, 6, 52138-52160.
4. **Aggour, K. S., Bonissone, P. P., Cheetham, W. E., & Messmer, R. P. (2006).** Automating the underwriting of insurance applications. *AI magazine*, 27(3), 36-36.
5. **Aitken, R. (2017).** 'All data is credit data': Constituting the unbanked. *Competition & Change*. 21(4), 274- 300.
6. **Akinyokun, O. C. (2015).** Fuzzy logic-driven expert system for the diagnosis of heart failure disease. *Artif. Intell. Research*, 12-21.
7. **Ala'raj, M., & Abbod, M. F. (2016).** Classifiers consensus system approach for credit scoring. *Knowledge-Based Systems*, 104, 89-105.
8. **Bensic, M., Sarlija, N., & Zekic- Susac, M. (2005).** Modelling small- business credit scoring by using logistic regression, neural networks and decision trees. *Intelligent Systems in Accounting, Finance & Management: International Journal*, 13(3), 133-150.
9. **Bijak, K., & Thomas, L. C. (2012).** Does segmentation always improve model performance in credit scoring? *Expert Systems with Applications*, 39(3), 2433-2442.
10. **Casalicchio, G., Molnar, C., & Bischl, B. (2018).** Visualizing the feature importance for black box models. *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, (pp. pp. 655-670).
11. **Cawley, G. C., & Talbot, N. L. (2010).** On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11(Jul), 2079-2107.
12. **Chen, W., Ma, C., & Ma, L. (2009).** Mining the customer credit using hybrid support vector machine technique. *Expert systems with applications*, 36(4), 7611-7616.
13. **Chen, Y. W., Yang, J. B., Xu, D. L., Zhou, Z. J., & Tang, D. W. (2011).** Inference analysis and adaptive training for belief rule based systems. *Expert Systems with Applications*, 38(10), 12845-12860.

13. **Cocheo, S. (1995).** Automated underwriting: Tower of techno-Babel? American Bankers Association.ABA Banking Journal, 87(9), 68. **Feigenbaum, E. A. (1980).** Knowledge engineering: The applied side of artificial intelligence. STANFORD UNIV CA DEPT OF COMPUTER SCIENCE. (No. STAN-CS-80-812). **Gates, S. W., Perry, V. G., & Zorn, P. M. (2002).** Automated underwriting in mortgage lending: good news for the underserved? Housing Policy Debate, 13(2), 369-391.
14. **Glassman, C. A., & Wilkins, H. M. (1997).** Credit scoring: probabilities and pitfalls. Journal of Retail Banking Services, 19(2), 53-57.
15. **Gomez, F., Hull, R., Karr, C., Hosken, B., & Verhagen, W. (1992).** Combining factual and heuristic knowledge in knowledge acquisition. Telematics and Informatics, 9(3-4), 297- 311.
16. **Harris, T. (2015).** Credit scoring using the clustered support vector machine. Expert Systems with Applications, 42(2), 741-750.
17. **Hens, A. B., & Tiwari, M. K. (2012).** Computational time reduction for credit scoring: An integrated approach based on support vector machine and stratified sampling method. Expert Systems with Applications, 39(8), 6774-6781. **Holsapple, C. W., Raj, V., & Wagner, W. P. (2008).** An experimental investigation of the impact of domain complexity on knowledge acquisition (KA) methods. Expert Systems with Applications, 35(3), 1084- 1094.
18. **Huang, C. L., Chen, M. C., & Wang, C. J. (2007).** Credit scoring with a data mining approach based on support vector machines. Expert systems with applications, 33(4), 847-856.
19. **Hung, C., & Chen, J. H. (2009).** A selective ensemble based on expected probabilities for bankruptcy prediction. Expert systems with applications, 36(3), 5297-5303. **Khashman, A. (2010).** Neural networks for credit risk evaluation: Investigation of different neural models and learning schemes. Expert Systems with Applications, 37(9), 6233- 6239
20. **Killin, J. (1992).** The management and maintenance of an operational KADS system development. International Conference on Knowledge Engineering and Knowledge Management (pp. 6-6). **Springer. Kim, Y. S., & Sohn, S. Y. (2004).** Managing loan customers using misclassification patterns of credit scoring model. Expert Systems with Applications, 26(4), 567-573.

- 21. Kingston, J. (1991).** X-MATE: Creating an interpretation model for credit risk assessment. In Research and development in expert systems VIII: proceedings of 11th annual technical conference of the BCS specialist group (pp. 165-174). Cambridge University Press. **Krovvidy, S. (2008).** Custom DU: A Web-Based Business User-Driven Automated Underwriting System. *AI Magazine*, 29(1), 41-41.
- 22. Lee, T. S., & Chen, I. F. (2005).** A two-stage hybrid credit scoring model using artificial neural networks and multivariate adaptive regression splines. *Expert Systems with Applications*, 28(4), 743-752. **Leong, C. K. (2016).** Credit risk scoring with Bayesian network models. *Computational Economics*, 47(3), 423-446. **Li, S. T., Shiue, W., & Huang, M. H. (2006).** The evaluation of consumer loans using support vector machines. *Expert Systems with Applications*, 30(4), 772-782. **Lipton, Z. C. (2016).** The mythos of model interpretability. *ICML Workshop on Human Interpretability in Machine Learning*.
- 23. Lundberg, S. M., & Lee, S. I. (2017).** A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems* (pp. 4765-4774). **Luo, C., Wu, D., & Wu, D. (2017).** A deep learning approach for credit scoring using credit default swaps. *Engineering Applications of Artificial Intelligence*, 65, 465-470. **Marqués, A. I., García, V., & Sánchez, J. S. (2012).** Exploring the behaviour of base classifiers in credit scoring ensembles. *Expert Systems with Applications*, 39(11), 10244-10250. **Metawa, N., Hassan, M. K., & Elhoseny, M. (2017).** Genetic algorithm-based model for optimizing bank lending decisions. *Expert Systems with Applications*, 80, 75-82.